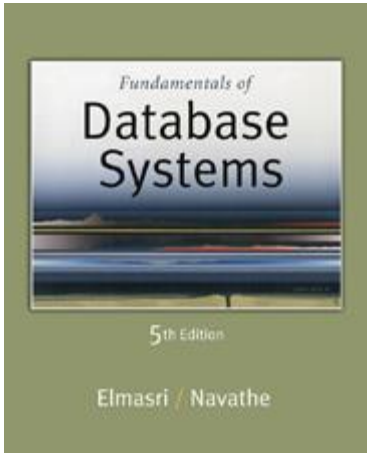
Chapter 3

Data Modeling Using the Entity-Relationship (ER) Model



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Chapter Outline

- Overview of Database Design Process
- Example Database Application (COMPANY)
- ER Model Concepts
  - Entities and Attributes
  - Entity Types, Value Sets, and Key Attributes
  - Relationships and Relationship Types
  - Weak Entity Types
  - Roles and Attributes in Relationship Types
- ER Diagrams - Notation
- ER Diagram for COMPANY Schema
- Alternative Notations – UML class diagrams, others

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Overview of Database Design Process

- Two main activities:
  - Database design
  - Applications design
- Focus in this chapter on database design
  - To design the conceptual schema for a database application
- Applications design focuses on the programs and interfaces that access the database
  - Generally considered part of software engineering

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Overview of Database Design Process

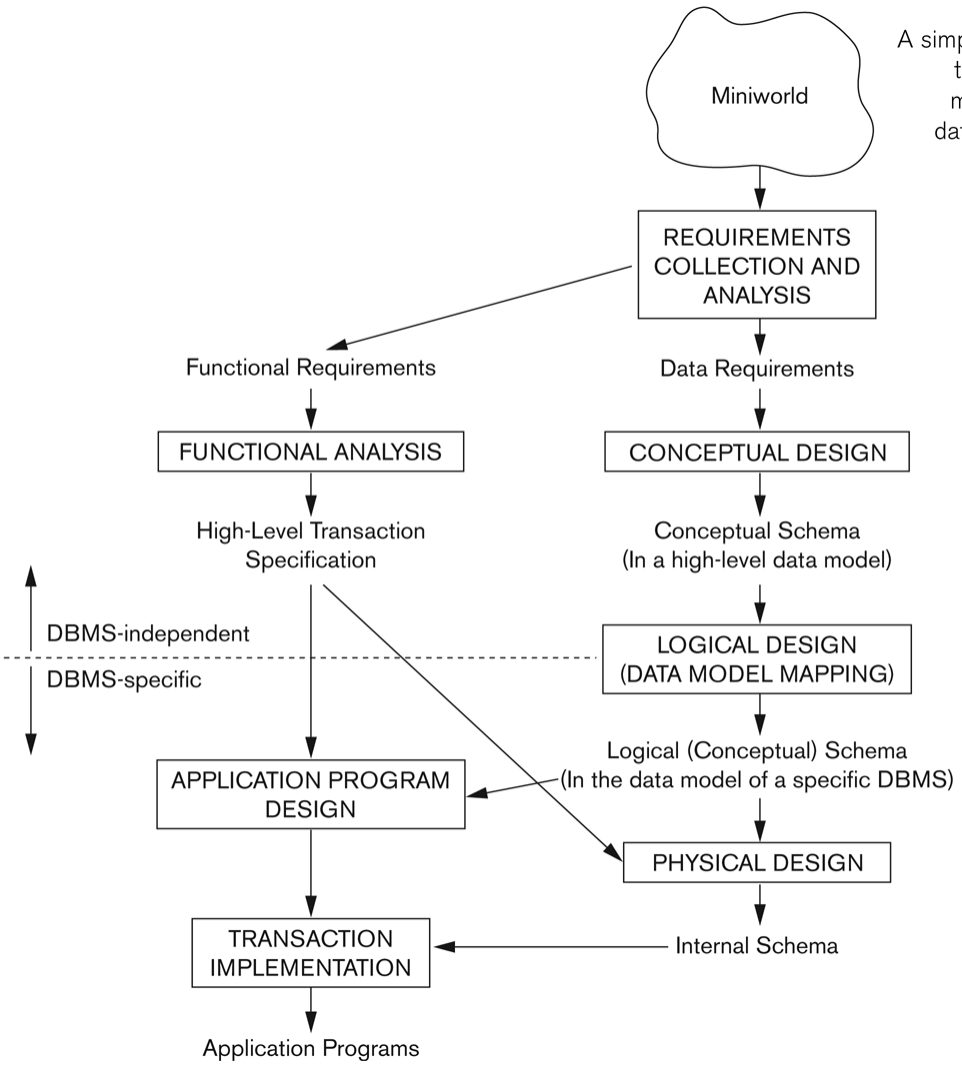


Figure 3.1
A simplified diagram to illustrate the main phases of database design.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example COMPANY Database

- We need to create a database schema design based on the following (simplified) **requirements** of the COMPANY Database:
  - The company is organized into DEPARTMENTs. Each department has a name, number and an employee who *manages* the department. We keep track of the start date of the department manager. A department may have several locations.
  - Each department *controls* a number of PROJECTs. Each project has a unique name, unique number and is located at a single location.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

The company is organised in departments. Each department is managed by one of its employees but an employee may manage one department at most. An employee may supervise one or more employees, but an employee is supervised by only one employee. Not every employee is a supervisor and not every employee is supervised. The database should keep track of the start date when an employee started managing a department. Each department has a name, a number and a phone

A department may control one or more projects. The data held on a project is a unique name and number, budget, and the date the project started and the finishing date. Each department employs more than one employee.

The data held on an employee is his name (first and last), address (Town, street, Post code), salary, gender, date of birth, qualifications, and NIN (National Insurance Number) which uniquely identifies each employee throughout the company. An employee is employed by one department but may work on several projects each of which is controlled by one department. The company would like to keep track of the number of hours per week that an employee works on each project. Each project has more than one employee working on it.

For insurance purposes the company would like to keep track of the dependents of employees. The database should store each dependent first name, sex, date of birth, and relationship to the employee. An employee may have one or more dependents and a dependent is a dependent of one employee only. None of the dependents' attributes can uniquely identify a dependent.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example COMPANY Database (Contd.)

- We store each EMPLOYEE's social security number, address, salary, sex, and birthdate.
  - Each employee *works for* one department but may *work on* several projects.
  - We keep track of the number of hours per week that an employee currently works on each project.
  - We also keep track of the *direct supervisor* of each employee.
- Each employee may *have* a number of DEPENDENTs.
  - For each dependent, we keep track of their name, sex, birthdate, and relationship to the employee.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

ER Model Concepts

- Entities and Attributes
  - Entities are specific objects or things in the mini-world that are represented in the database.
    - For example the EMPLOYEE John Smith, the Research DEPARTMENT, the ProductX PROJECT
  - Attributes are properties used to describe an entity.
    - For example an EMPLOYEE entity may have the attributes Name, SSN, Address, Sex, BirthDate
  - A specific entity will have a value for each of its attributes.
    - For example a specific employee entity may have Name='John Smith', SSN='123456789', Address ='731, Fondren, Houston, TX', Sex='M', BirthDate='09-JAN-55‘
  - Each attribute has a *value set* (or data type) associated with it – e.g. integer, string, subrange, enumerated type, …

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Types of Attributes (1)

- Simple
  - Each entity has a single atomic value for the attribute. For example, SSN or Sex.
- Composite
  - The attribute may be composed of several components. For example:
    - Address(Apt#, House#, Street, City, State, ZipCode, Country), or
    - Name(FirstName, MiddleName, LastName).
    - Composition may form a hierarchy where some components are themselves composite.
- Multi-valued
  - An entity may have multiple values for that attribute. For example, Color of a CAR or PreviousDegrees of a STUDENT.
    - Denoted as {Color} or {PreviousDegrees}.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Types of Attributes (2)

- In general, composite and multi-valued attributes may be nested arbitrarily to any number of levels, although this is rare.
  - For example, PreviousDegrees of a STUDENT is a composite multi-valued attribute denoted by {PreviousDegrees (College, Year, Degree, Field)}
  - Multiple PreviousDegrees values can exist
  - Each has four subcomponent attributes:
    - College, Year, Degree, Field

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example of a composite attribute

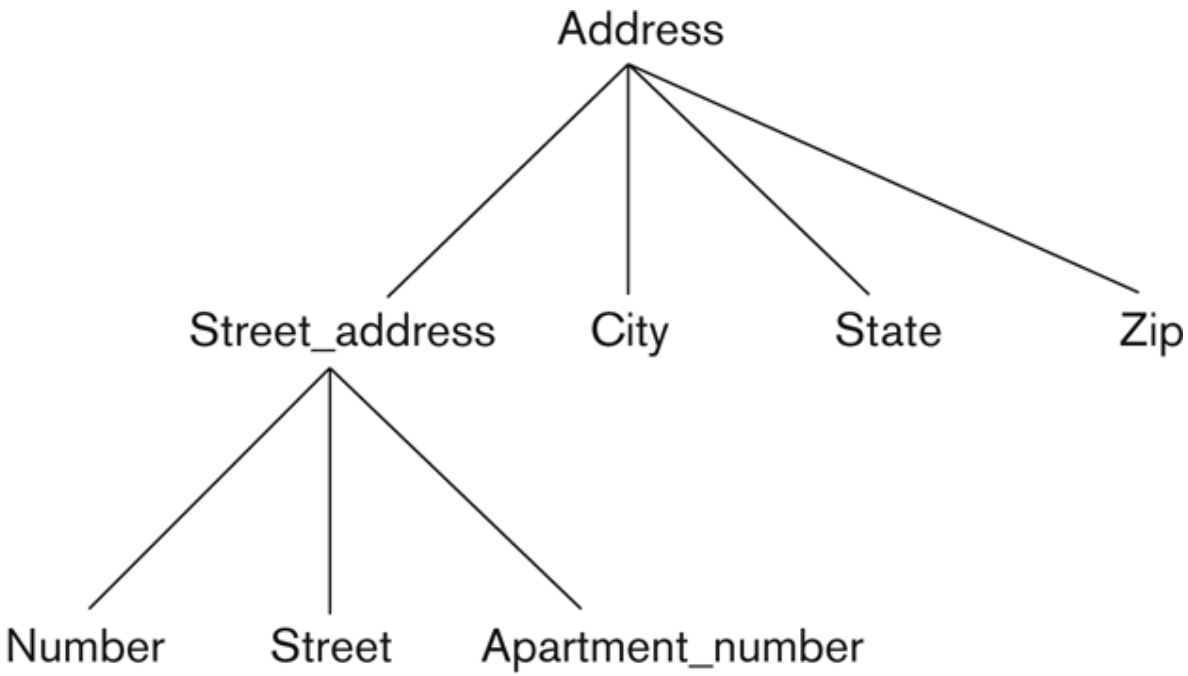


Figure 3.4
A hierarchy of composite attributes.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Entity Types and Key Attributes (1)

- Entities with the same basic attributes are grouped or typed into an entity type.
  - For example, the entity type EMPLOYEE and PROJECT.
- An attribute of an entity type for which each entity must have a unique value is called a key attribute of the entity type.
  - For example, SSN of EMPLOYEE.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Entity Types and Key Attributes (2)

- A key attribute may be composite.
  - VehicleTagNumber is a key of the CAR entity type with components (Number, State).
- An entity type may have more than one key.
  - The CAR entity type may have two keys:
    - VehicleIdentificationNumber (popularly called VIN)
    - VehicleTagNumber (Number, State), aka license plate number.
- Each key is <u>underlined</u>

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Displaying an Entity type

- In ER diagrams, an entity type is displayed in a rectangular box
- Attributes are displayed in ovals
  - Each attribute is connected to its entity type
  - Components of a composite attribute are connected to the oval representing the composite attribute
  - Each key attribute is underlined
  - Multivalued attributes displayed in double ovals
- See CAR example on next slide

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Entity Type CAR with two keys and a corresponding Entity Set

(a)

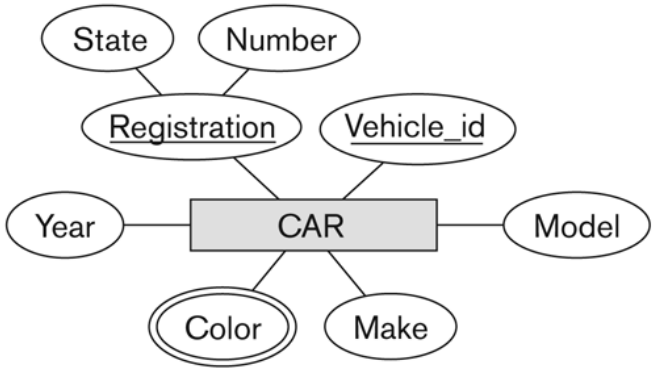


Figure 3.7
The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

(b)

CAR
Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR_1
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR_2
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR_3
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

⋮

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Entity Set

- Each entity type will have a collection of entities stored in the database
  - Called the **entity set**
- Previous slide shows three CAR entity instances in the entity set for CAR
- Same name (CAR) used to refer to both the entity type and the entity set
- Entity set is the current *state* of the entities of that type that are stored in the database

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Initial Design of Entity Types for the COMPANY Database Schema

- Based on the requirements, we can identify four initial entity types in the COMPANY database:
  - DEPARTMENT
  - PROJECT
  - EMPLOYEE
  - DEPENDENT
- Their initial design is shown on the following slide
- The initial attributes shown are derived from the requirements description

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Initial Design of Entity Types:
EMPLOYEE, DEPARTMENT, PROJECT, DEPENDENT

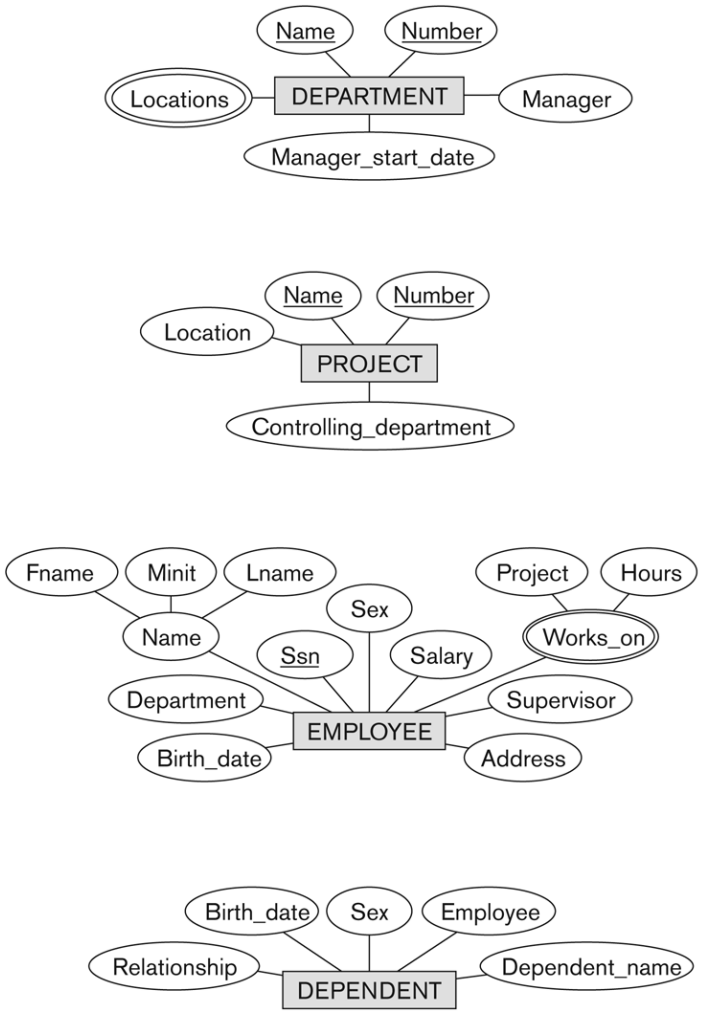


Figure 3.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Refining the initial design by introducing **relationships**

- The initial design is typically not complete
- Some aspects in the requirements will be represented as **relationships**
- ER model has three main concepts:
  - Entities (and their entity types and entity sets)
  - Attributes (simple, composite, multivalued)
  - Relationships (and their relationship types and relationship sets)
- We introduce relationship concepts next

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationships and Relationship Types (1)

- A **relationship** relates two or more distinct entities with a specific meaning.
  - For example, EMPLOYEE John Smith *works on* the ProductX PROJECT, or EMPLOYEE Franklin Wong *manages* the Research DEPARTMENT.
- Relationships of the same type are grouped or typed into a **relationship type**.
  - For example, the WORKS_ON relationship type in which EMPLOYEEs and PROJECTs participate, or the MANAGES relationship type in which EMPLOYEEs and DEPARTMENTs participate.
- The degree of a relationship type is the number of participating entity types.
  - Both MANAGES and WORKS_ON are *binary* relationships.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationship instances of the WORKS_FOR N:1 relationship between EMPLOYEE and DEPARTMENT

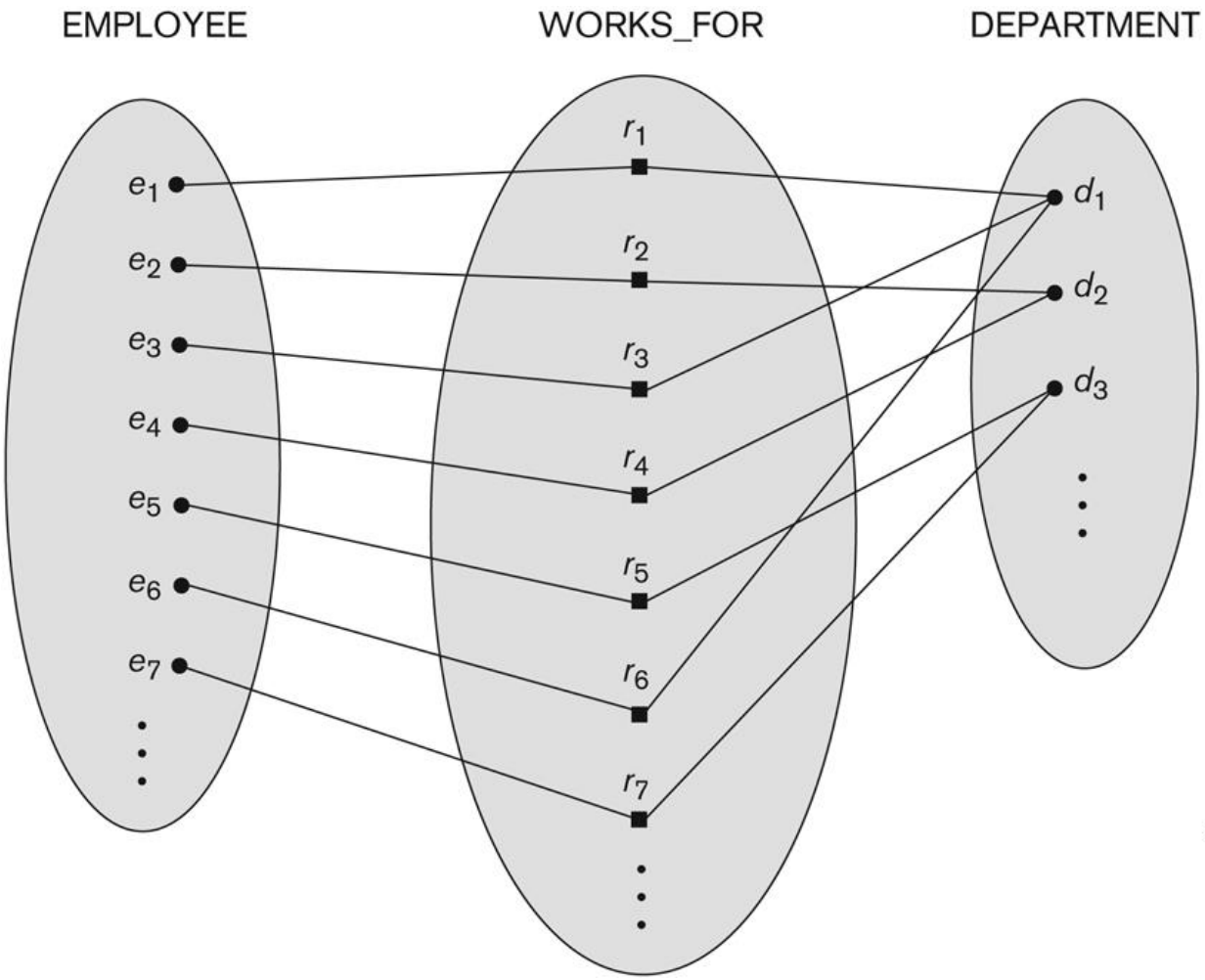


Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationship instances of the M:N WORKS_ON relationship between EMPLOYEE and PROJECT

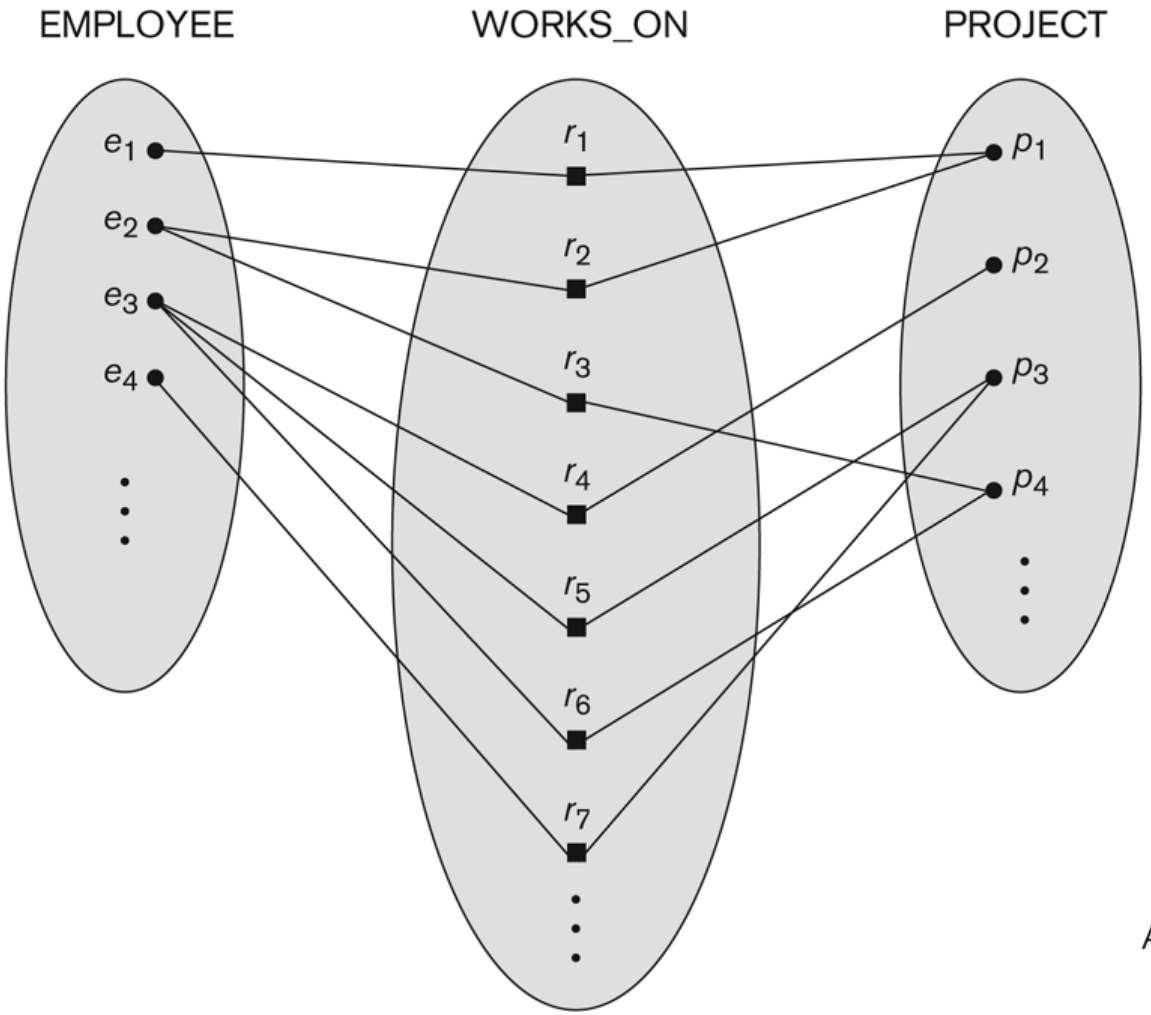


Figure 3.13
An M:N relationship, WORKS_ON.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationship type vs. relationship set (1)

- Relationship Type:
  - Is the schema description of a relationship
  - Identifies the relationship name and the participating entity types
  - Also identifies certain relationship constraints
- Relationship Set:
  - The current set of relationship instances represented in the database
  - The current *state* of a relationship type

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationship type vs. relationship set (2)

- Previous figures displayed the relationship sets
- Each instance in the set relates individual participating entities – one from each participating entity type
- In ER diagrams, we represent the *relationship type* as follows:
  - Diamond-shaped box is used to display a relationship type
  - Connected to the participating entity types via straight lines

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Refining the COMPANY database schema by introducing relationships

- By examining the requirements, six relationship types are identified
- All are *binary* relationships(degree 2)
- Listed below with their participating entity types:
  - WORKS_FOR (between EMPLOYEE, DEPARTMENT)
  - MANAGES (also between EMPLOYEE, DEPARTMENT)
  - CONTROLS (between DEPARTMENT, PROJECT)
  - WORKS_ON (between EMPLOYEE, PROJECT)
  - SUPERVISION (between EMPLOYEE (as subordinate), EMPLOYEE (as supervisor))
  - DEPENDENTS_OF (between EMPLOYEE, DEPENDENT)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

ER DIAGRAM – Relationship Types are:

WORKS_FOR, MANAGES, WORKS_ON, CONTROLS, SUPERVISION, DEPENDENTS_OF

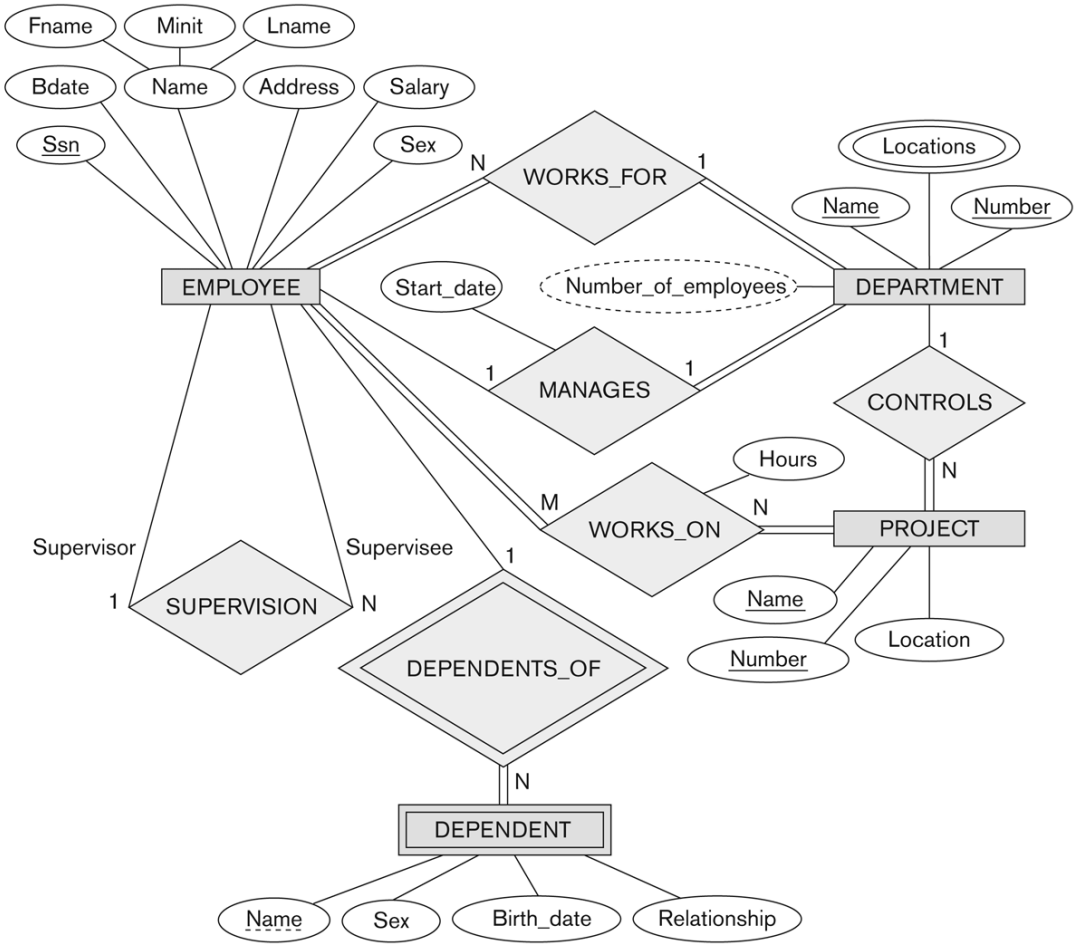


Figure 3.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Discussion on Relationship Types

- In the refined design, some attributes from the initial entity types are refined into relationships:
  - Manager of DEPARTMENT -> MANAGES
  - Works_on of EMPLOYEE -> WORKS_ON
  - Department of EMPLOYEE -> WORKS_FOR
  - etc
- In general, more than one relationship type can exist between the same participating entity types
  - MANAGES and WORKS_FOR are distinct relationship types between EMPLOYEE and DEPARTMENT
  - Different meanings and different relationship instances.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Recursive Relationship Type

- An relationship type whose with the same participating entity type in **distinct roles**
- Example: the SUPERVISION relationship
- EMPLOYEE participates twice in two distinct roles:
  - supervisor (or boss) role
  - supervisee (or subordinate) role
- Each relationship instance relates two distinct EMPLOYEE entities:
  - One employee in *supervisor* role
  - One employee in *supervisee* role

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Weak Entity Types

- An entity that does not have a key attribute
- A weak entity must participate in an identifying relationship type with an owner or identifying entity type
- Entities are identified by the combination of:
  - A partial key of the weak entity type
  - The particular entity they are related to in the identifying entity type
- **Example:**
  - A DEPENDENT entity is identified by the dependent's first name, *and* the specific EMPLOYEE with whom the dependent is related
  - Name of DEPENDENT is the *partial key*
  - DEPENDENT is a *weak entity type*
  - EMPLOYEE is its identifying entity type via the identifying relationship type DEPENDENT_OF

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Constraints on Relationships

- Constraints on Relationship Types
  - (Also known as ratio constraints)
  - Cardinality Ratio (specifies *maximum* participation)
    - One-to-one (1:1)
    - One-to-many (1:N) or Many-to-one (N:1)
    - Many-to-many (M:N)
  - Existence Dependency Constraint (specifies *minimum* participation) (also called participation constraint)
    - zero (optional participation, not existence-dependent)
    - one or more (mandatory participation, existence-dependent)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Many-to-one (N:1) Relationship

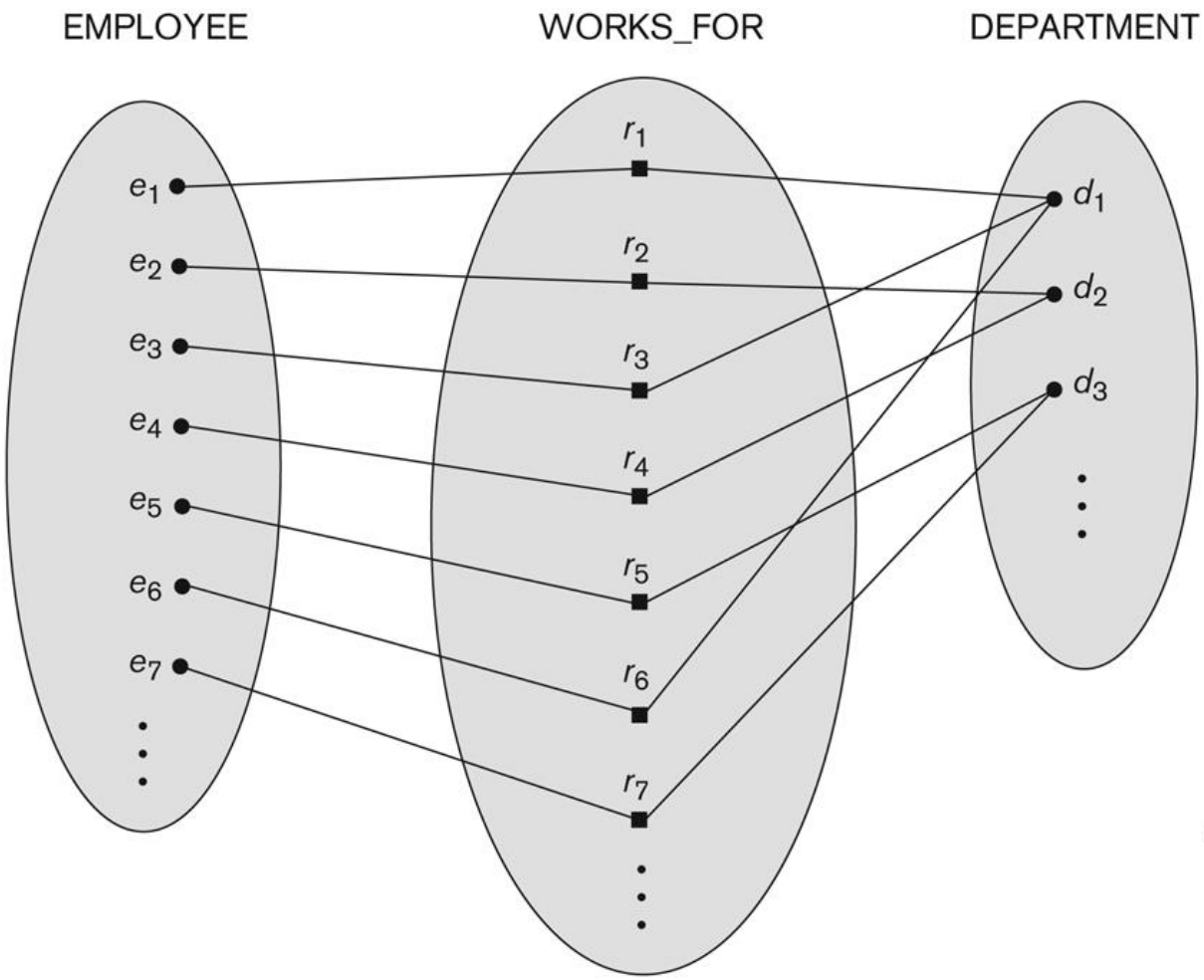


Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Many-to-many (M:N) Relationship

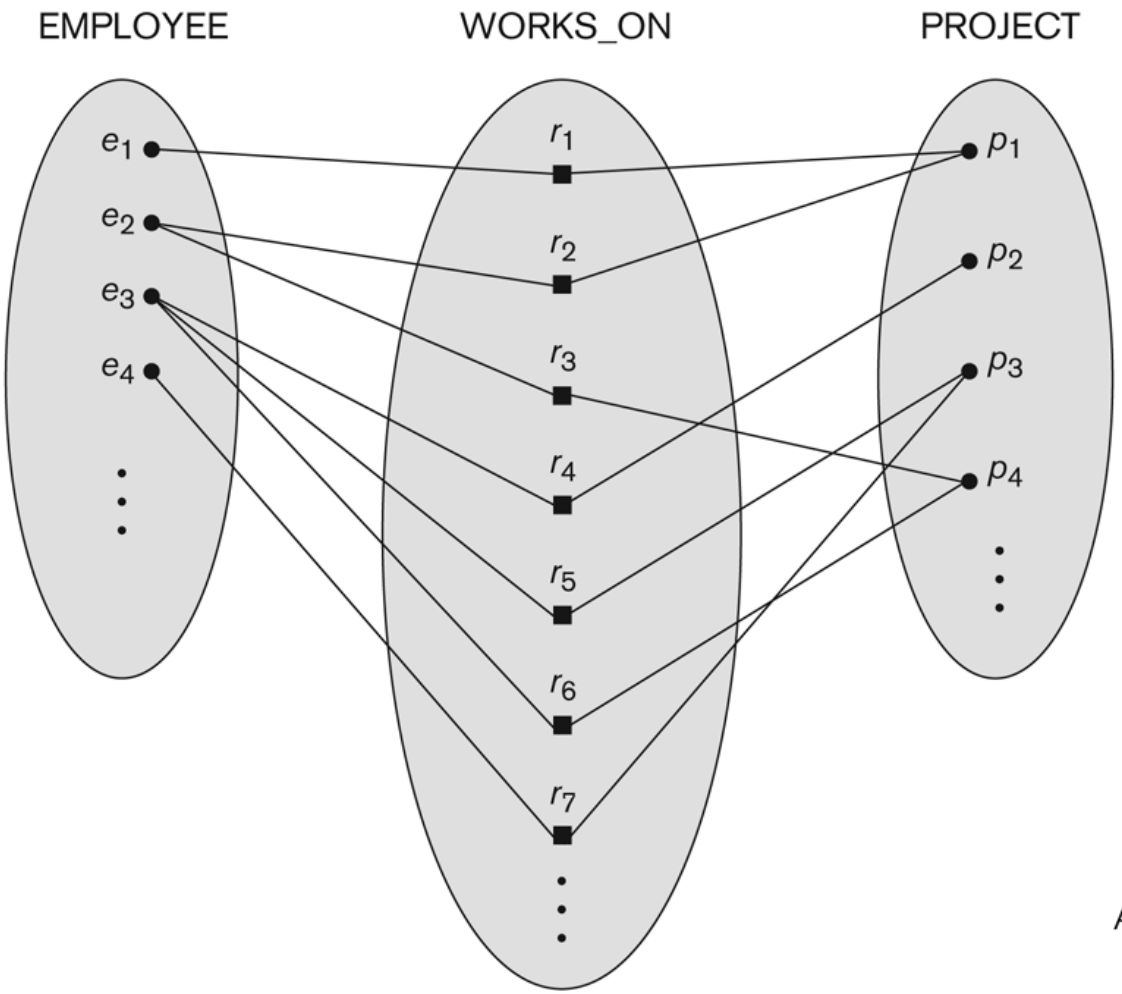


Figure 3.13
An M:N relationship, WORKS_ON.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Displaying a recursive relationship

- In a recursive relationship type.
  - Both participations are same entity type in different roles.
  - For example, SUPERVISION relationships between EMPLOYEE (in role of supervisor or boss) and (another) EMPLOYEE (in role of subordinate or worker).
- In following figure, first role participation labeled with 1 and second role participation labeled with 2.
- In ER diagram, need to display role names to distinguish participations.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

A Recursive Relationship Supervision`

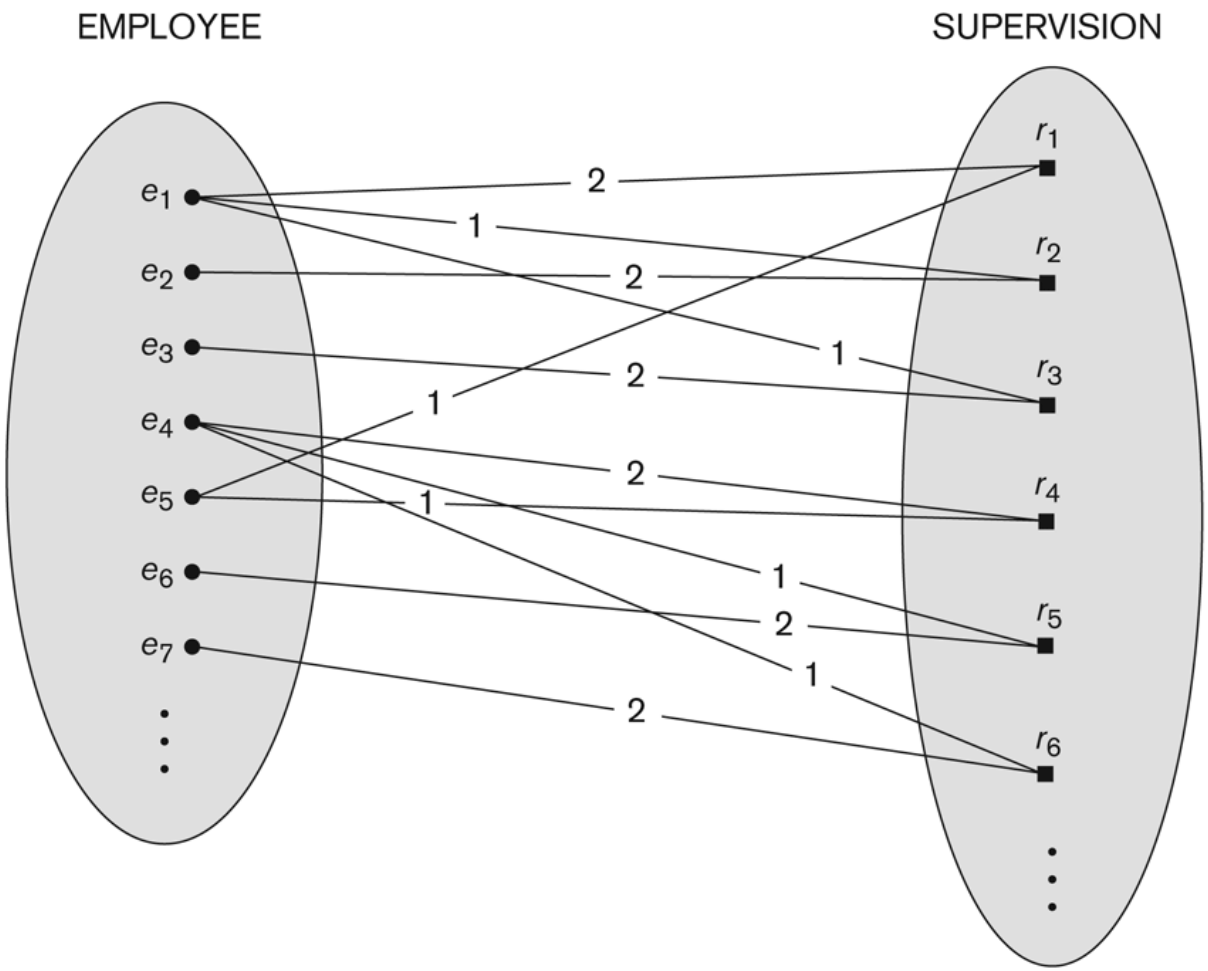


Figure 3.11
A recursive relationship SUPERVISION between EMPLOYEE in the *supervisor* role (1) and EMPLOYEE in the *subordinate* role (2).

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Recursive Relationship Type is: SUPERVISION (participation role names are shown)

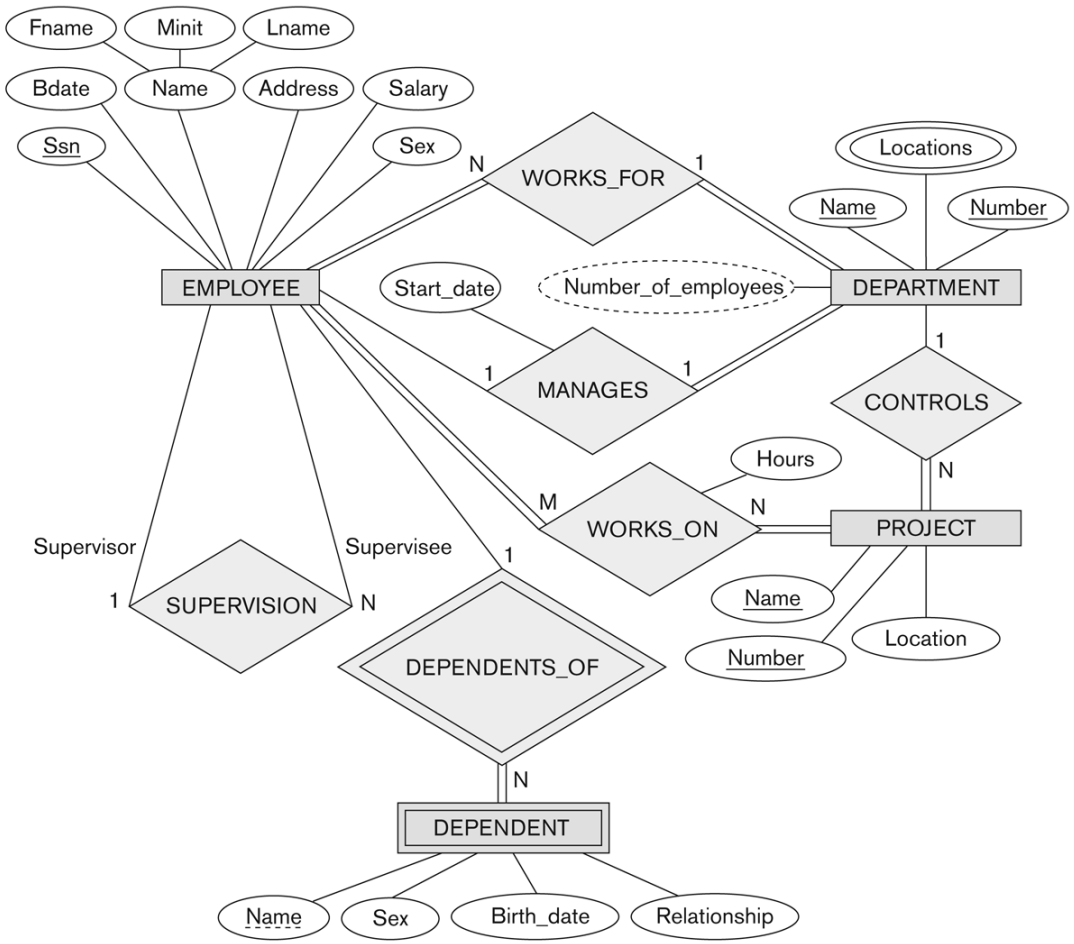


Figure 3.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Attributes of Relationship types

- A relationship type can have attributes:
  - For example, HoursPerWeek of WORKS_ON
  - Its value for each relationship instance describes the number of hours per week that an EMPLOYEE works on a PROJECT.
    - A value of HoursPerWeek depends on a particular (employee, project) combination
  - Most relationship attributes are used with M:N relationships
    - In 1:N relationships, they can be transferred to the entity type on the N-side of the relationship

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example Attribute of a Relationship Type: Hours of WORKS_ON

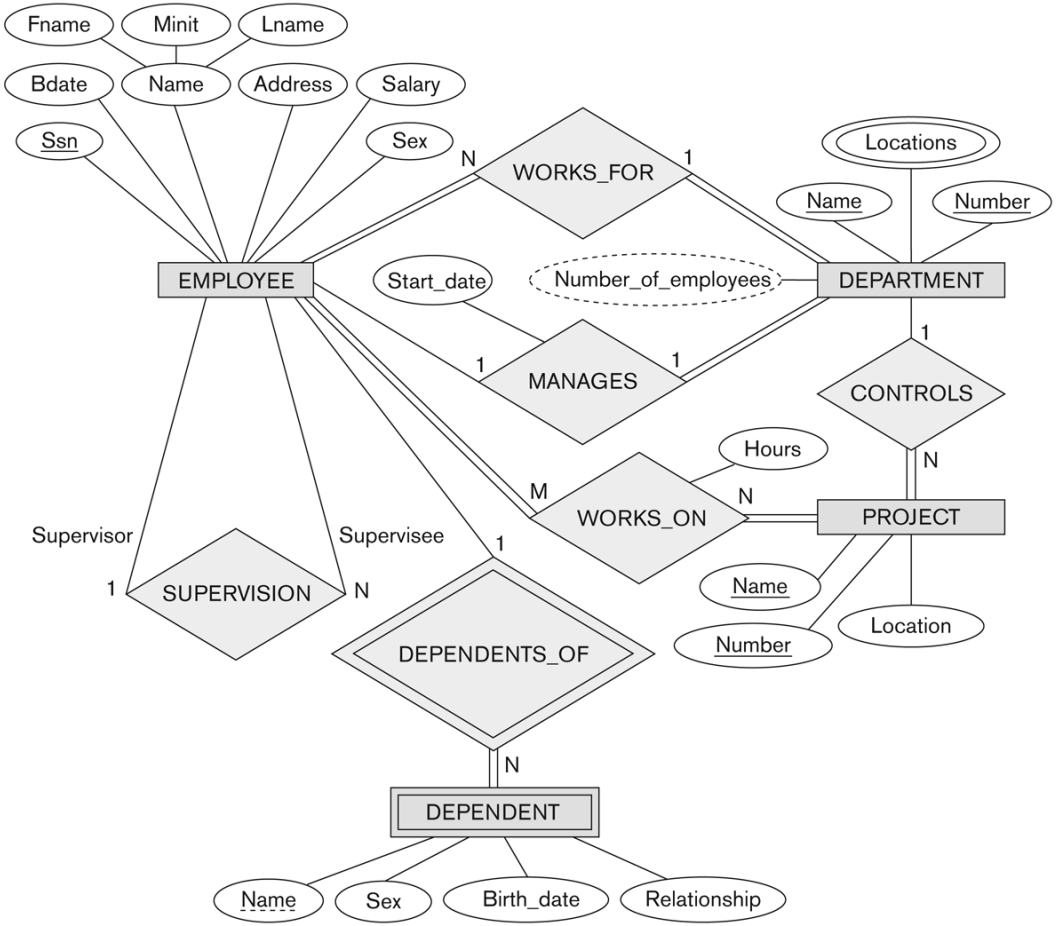


Figure 3.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Notation for Constraints on Relationships

- Cardinality ratio (of a binary relationship): 1:1, 1:N, N:1, or M:N
  - Shown by placing appropriate numbers on the relationship edges.
- Participation constraint (on each participating entity type): total (called existence dependency) or partial.
  - Total shown by double line, partial by single line.
- NOTE: These are easy to specify for Binary Relationship Types.

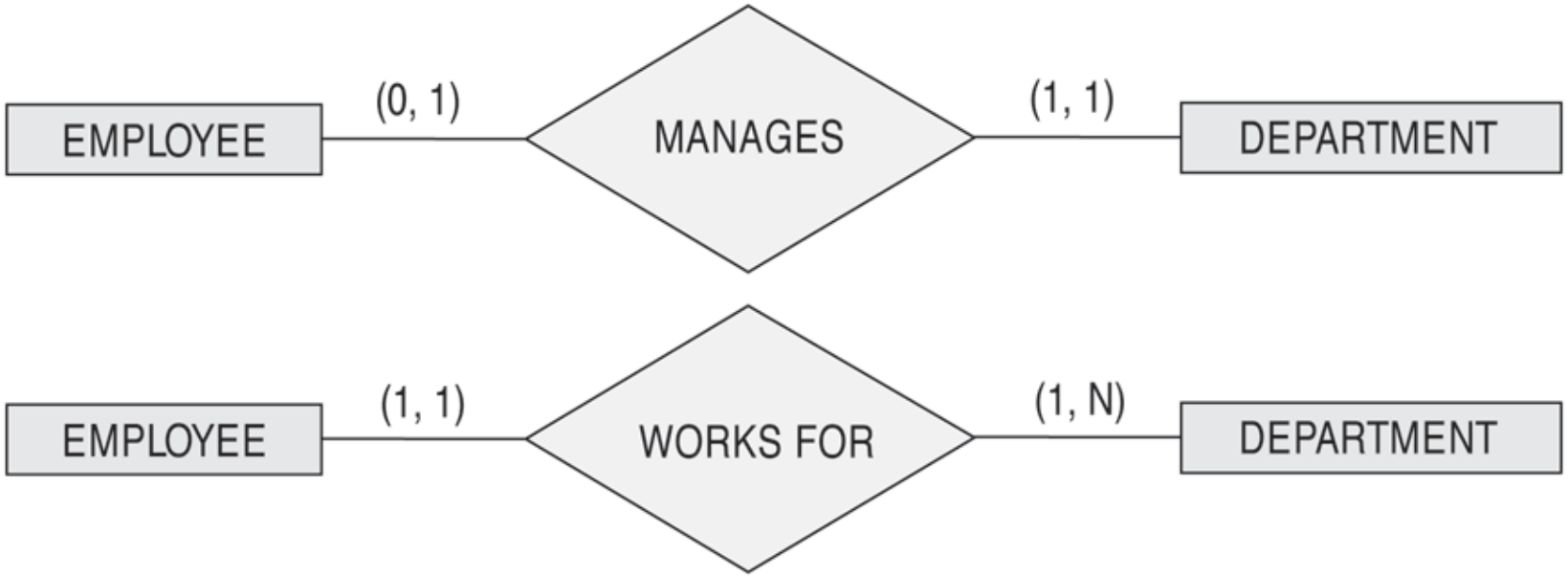
Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Alternative (min, max) notation for relationship structural constraints:

- Specified on each participation of an entity type E in a relationship type R
- Specifies that each entity e in E participates in at least *min* and at most *max* relationship instances in R
- Default(no constraint): min=0, max=n (signifying no limit)
- Must have min≤max, min≥0, max ≥1
- Derived from the knowledge of mini-world constraints
- Examples:
  - A department has exactly one manager and an employee can manage at most one department.
    - Specify (0,1) for participation of EMPLOYEE in MANAGES
    - Specify (1,1) for participation of DEPARTMENT in MANAGES
  - An employee can work for exactly one department but a department can have any number of employees.
    - Specify (1,1) for participation of EMPLOYEE in WORKS_FOR
    - Specify (0,n) for participation of DEPARTMENT in WORKS_FOR

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

The (min,max) notation for relationship constraints

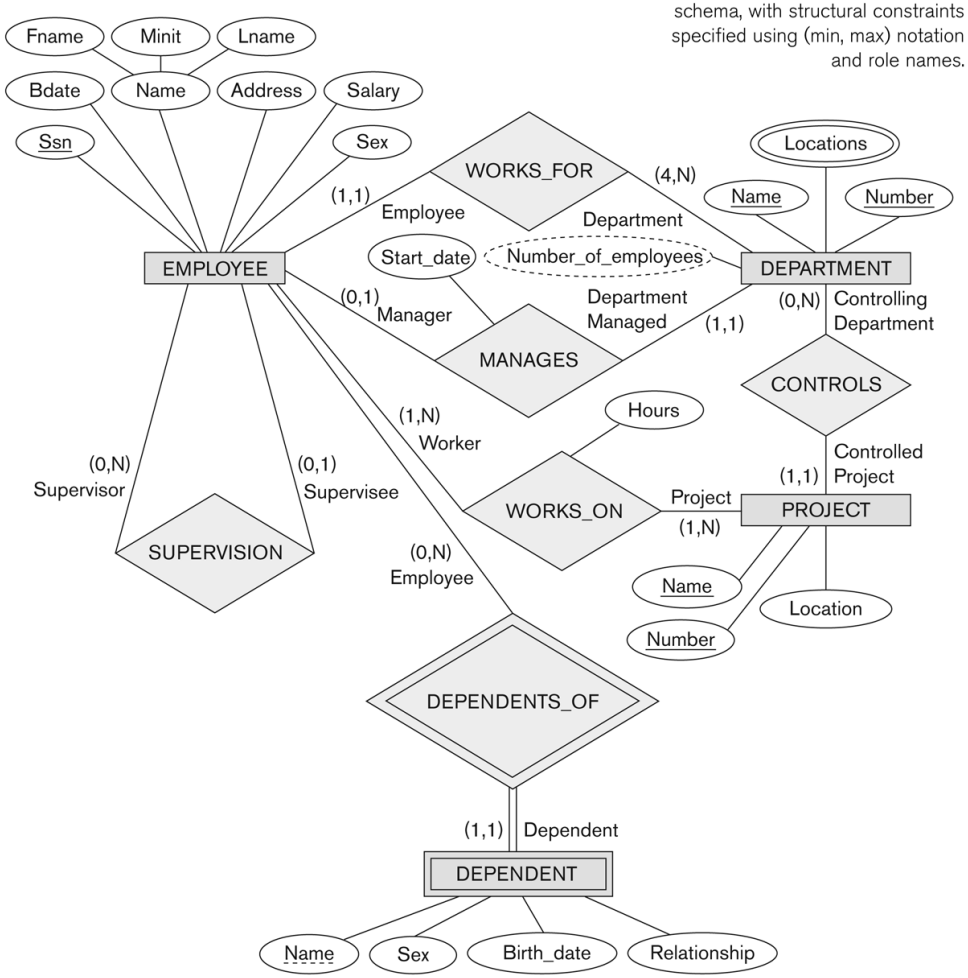


Read the min,max numbers next to the entity type and looking **away from** the entity type

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

COMPANY ER Schema Diagram using (min, max) notation

Figure 3.15
ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

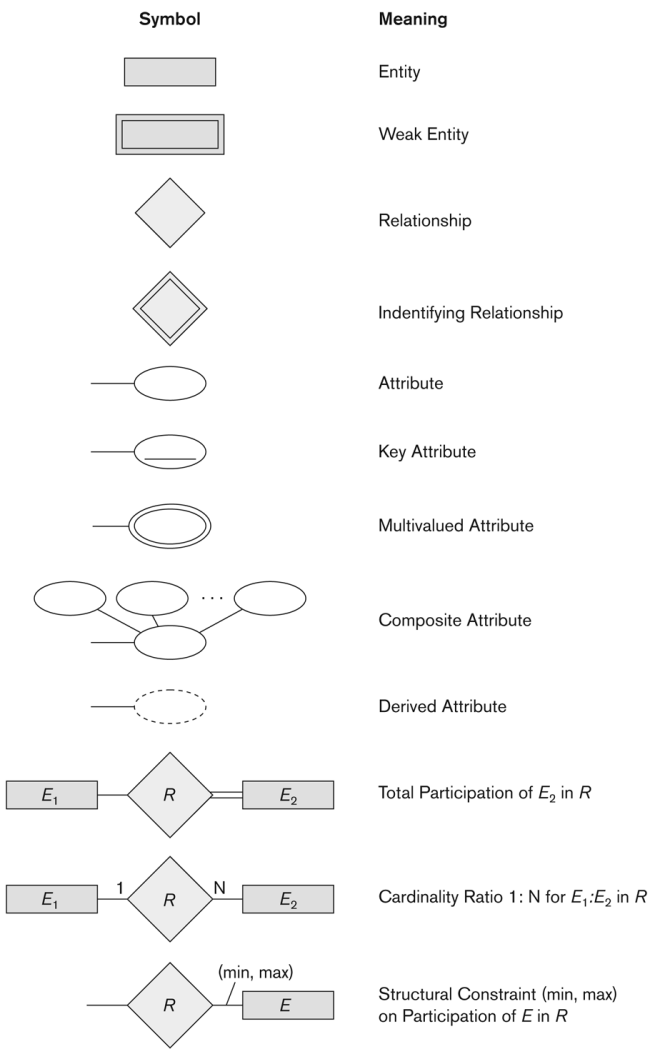
Alternative diagrammatic notation

- ER diagrams is one popular example for displaying database schemas
- Many other notations exist in the literature and in various database design and modeling tools
- Appendix A illustrates some of the alternative notations that have been used
- UML class diagrams is representative of another way of displaying ER concepts that is used in several commercial design tools

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Summary of notation for ER diagrams

Figure 3.14
Summary of the notation for ER diagrams.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

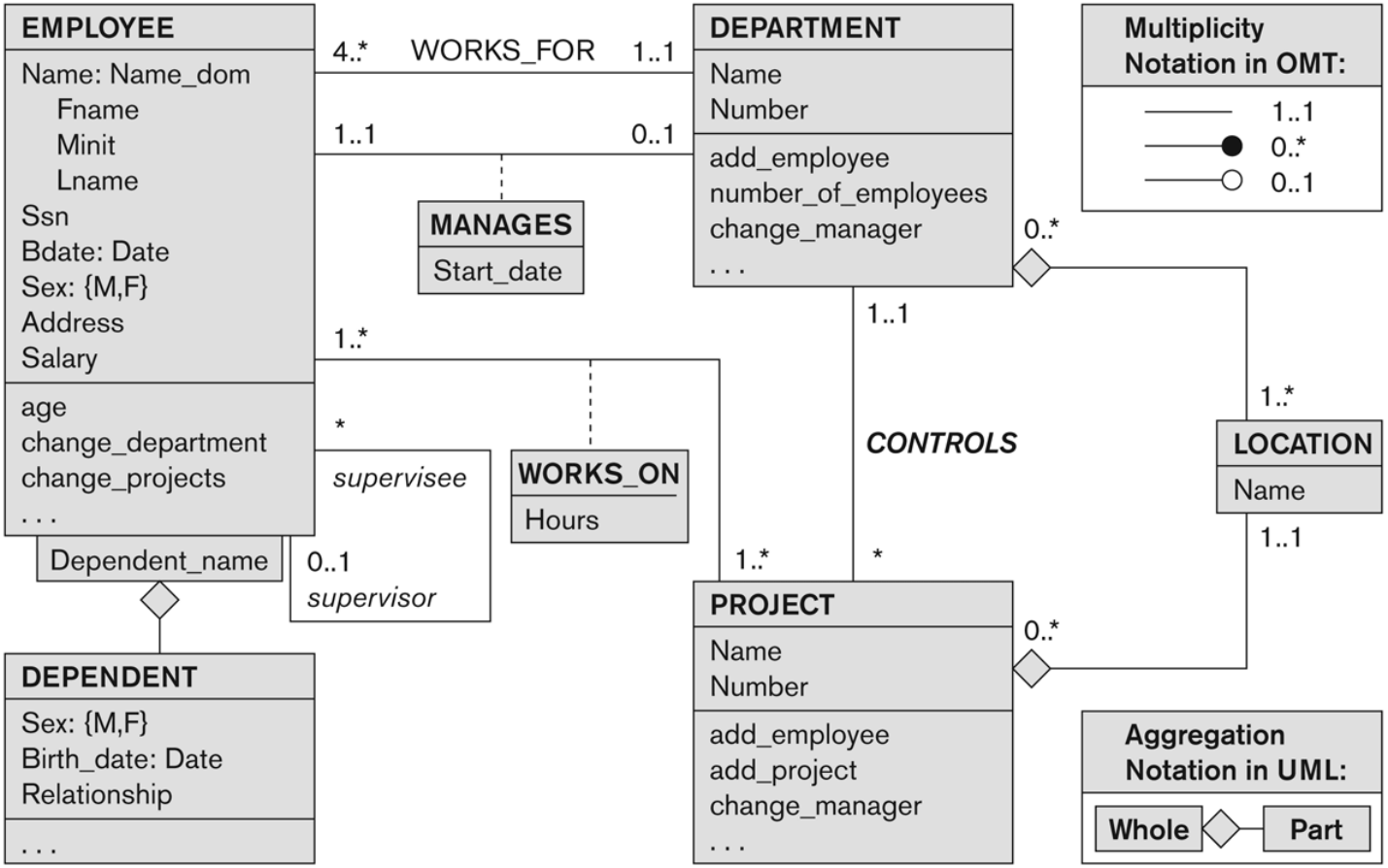
UML class diagrams

- Represent classes (similar to entity types) as large rounded boxes with three sections:
  - Top section includes entity type (class) name
  - Second section includes attributes
  - Third section includes class operations (operations are not in basic ER model)
- Relationships (called associations) represented as lines connecting the classes
  - Other UML terminology also differs from ER terminology
- Used in database design and object-oriented software design
- UML has many other types of diagrams for software design (see Chapter 12)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

UML class diagram for COMPANY database schema

Figure 3.16
The COMPANY conceptual schema in UML class diagram notation.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Other alternative diagrammatic notations

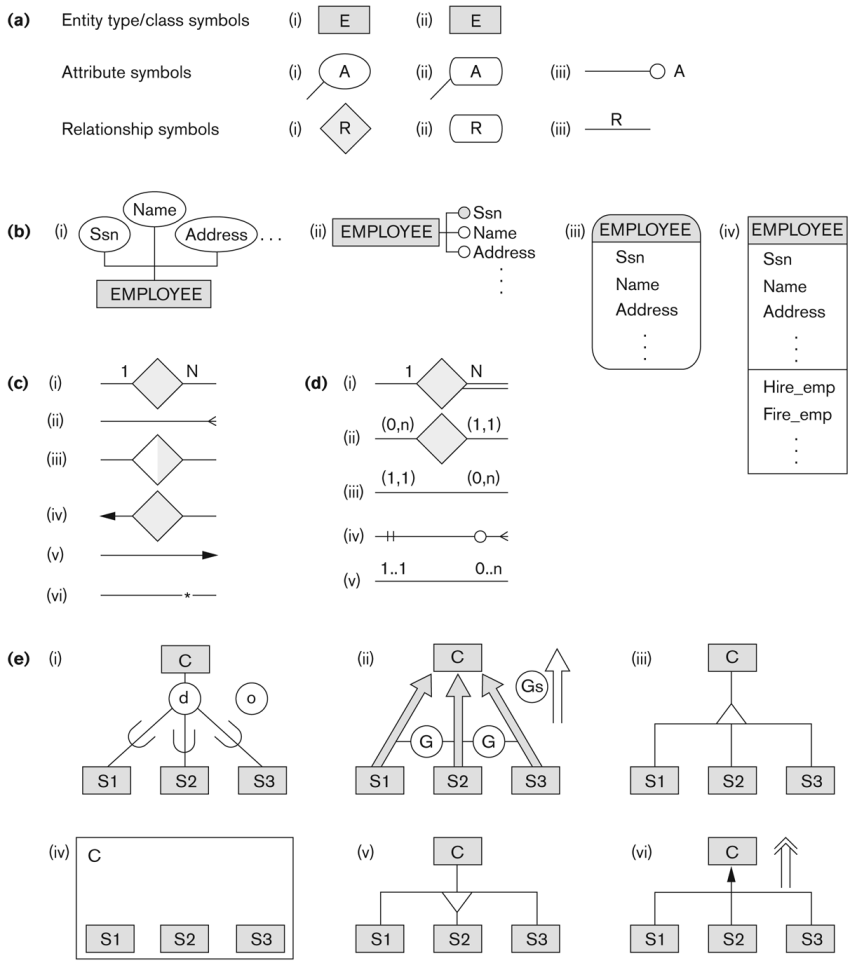


Figure A.1
Alternative notations. (a) Symbols for entity type/class, attribute, and relationship. (b) Displaying attributes. (c) Displaying cardinality ratios. (d) Various (min, max) notations. (e) Notations for displaying specialization/generalization.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Relationships of Higher Degree

- Relationship types of degree 2 are called binary
- Relationship types of degree 3 are called ternary and of degree n are called n-ary
- In general, an n-ary relationship is not equivalent to n binary relationships
- Constraints are harder to specify for higher-degree relationships (n > 2) than for binary relationships

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Discussion of n-ary relationships (n > 2)

- In general, 3 binary relationships can represent different information than a single ternary relationship (see Figure 3.17a and b on next slide)
- If needed, the binary and n-ary relationships can all be included in the schema design (see Figure 3.17a and b, where all relationships convey different meanings)
- In some cases, a ternary relationship can be represented as a weak entity if the data model allows a weak entity type to have multiple identifying relationships (and hence multiple owner entity types) (see Figure 3.17c)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Example of a ternary relationship

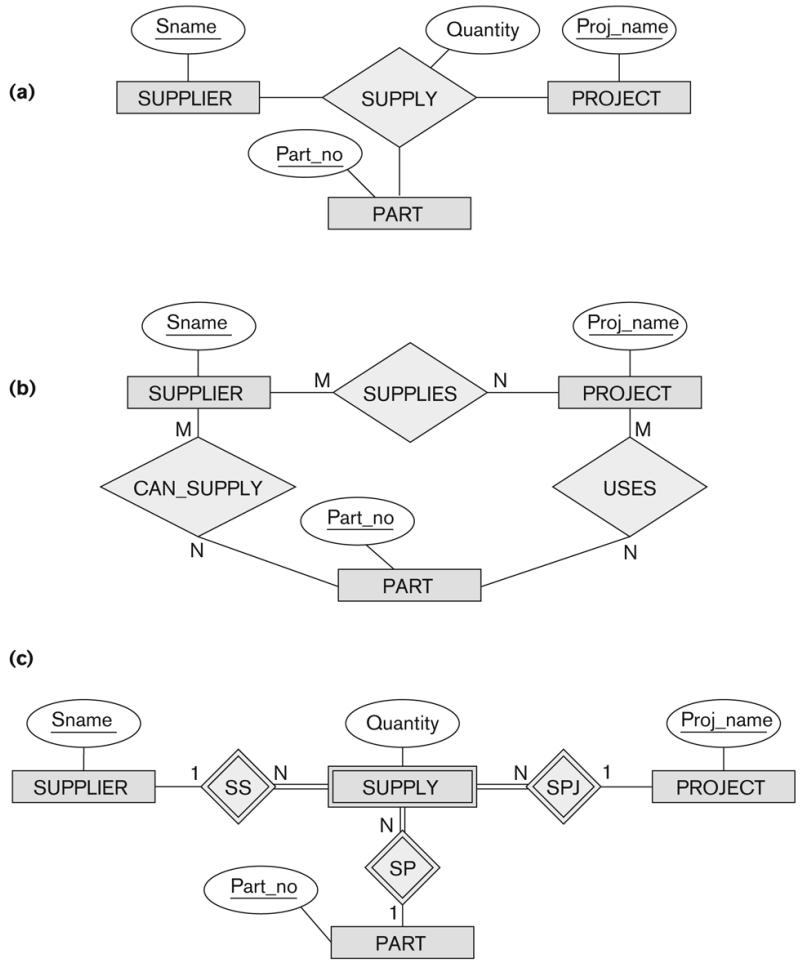


Figure 3.17
Ternary relationship types. (a) The SUPPLY relationship. (b) Three binary relationships not equivalent to SUPPLY. (c) SUPPLY represented as a weak entity type.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

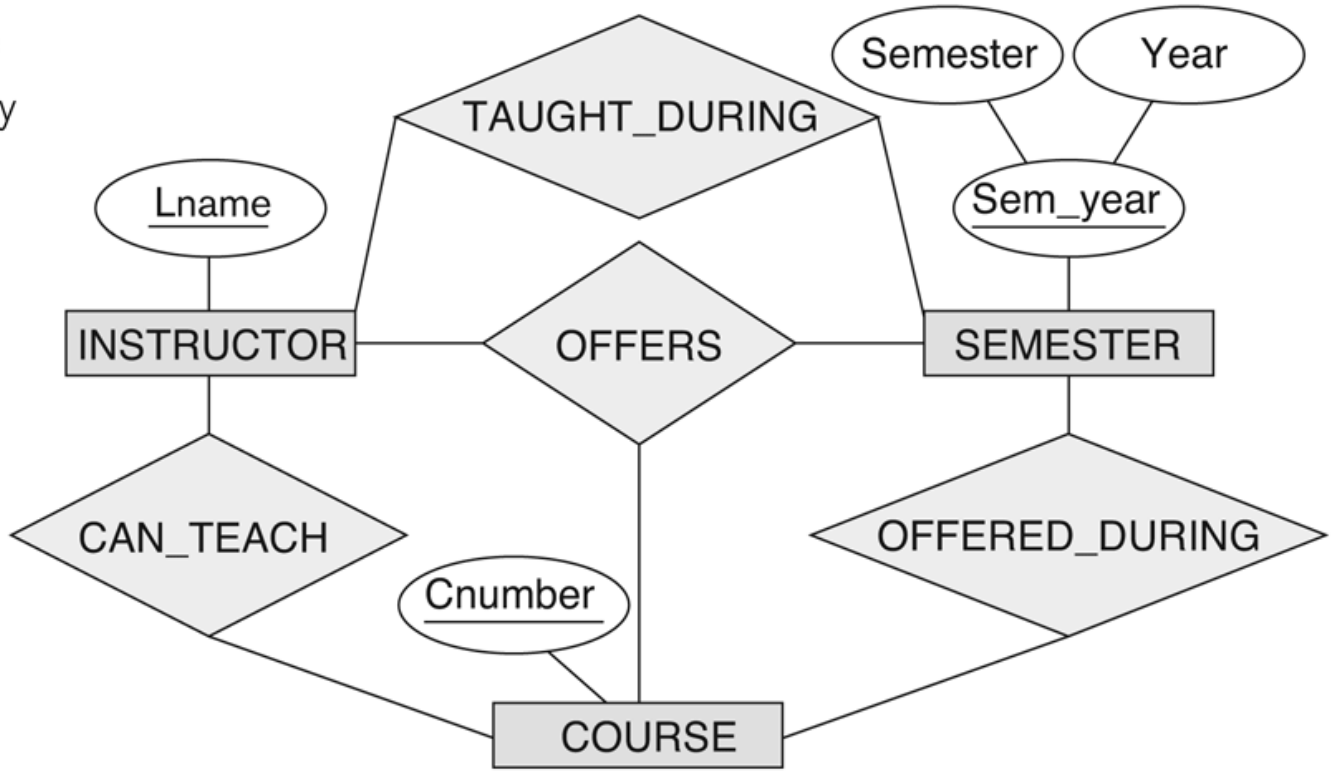
Discussion of n-ary relationships (n > 2)

- If a particular binary relationship can be derived from a higher-degree relationship at all times, then it is redundant
- For example, the TAUGHT_DURING binary relationship in Figure 3.18 (see next slide) can be derived from the ternary relationship OFFERS (based on the meaning of the relationships)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Another example of a ternary relationship

Figure 3.18
Another example of ternary versus binary relationship types.



Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Displaying constraints on higher-degree relationships

- The (min, max) constraints can be displayed on the edges – however, they do not fully describe the constraints
- Displaying a 1, M, or N indicates additional constraints
  - An M or N indicates no constraint
  - A 1 indicates that an entity can participate in at most one relationship instance *that has a particular combination of the other participating entities*
- In general, both (min, max) and 1, M, or N are needed to describe fully the constraints

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Data Modeling Tools

- A number of popular tools that cover conceptual modeling and mapping into relational schema design.
  - Examples: ERWin, S- Designer (Enterprise Application Suite), ER- Studio, etc.
- POSITIVES:
  - Serves as documentation of application requirements, easy user interface - mostly graphics editor support
- NEGATIVES:
  - Most tools lack a proper distinct notation for relationships with relationship attributes
  - Mostly represent a relational design in a diagrammatic form rather than a conceptual ER-based design

(See Chapter 12 for details)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Some of the Currently Available Automated Database Design Tools

COMPANY	TOOL	FUNCTIONALITY
Embarcadero Technologies	ER Studio	Database Modeling in ER and IDEF1X
	DB Artisan	Database administration, space and security management
Oracle	Developer 2000/Designer 2000	Database modeling, application development
Popkin Software	System Architect 2001	Data modeling, object modeling, process modeling, structured analysis/design
Platinum (Computer Associates)	Enterprise Modeling Suite: Erwin, BPWin, Paradigm Plus	Data, process, and business component modeling
Persistence Inc.	Pwertier	Mapping from O-O to relational model
Rational (IBM)	Rational Rose	UML Modeling & application generation in C++/JAVA
Resolution Ltd.	Xcase	Conceptual modeling up to code maintenance
Sybase	Enterprise Application Suite	Data modeling, business logic modeling
Visio	Visio Enterprise	Data modeling, design/reengineering Visual Basic/C++

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Extended Entity-Relationship (EER) Model (in next chapter)

- The entity relationship model in its original form did not support the specialization and generalization abstractions
- Next chapter illustrates how the ER model can be extended with
  - Type-subtype and set-subset relationships
  - Specialization/Generalization Hierarchies
  - Notation to display them in EER diagrams

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Chapter Summary

- ER Model Concepts: Entities, attributes, relationships
- Constraints in the ER model
- Using ER in step-by-step conceptual schema design for the COMPANY database
- ER Diagrams - Notation
- Alternative Notations – UML class diagrams, others

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is the difference between Entity Type and Entity Instance?**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is the difference between Entity Type and Entity Instance?**
- **Answer:**
- **Entity Type** = The category/template (like a form)
  Entity Instance = Actual data (filled form)

  - **Entity Type:** USER
  - **Entity Instances:**
  - User 1: "John Smith, john@email.com, joined 2023"

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is a Strong Entity with example?**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is a Strong Entity with example?**
- **Answer:**
- **Strong Entity** = Can exist on its own, has its own unique key
- **Characteristics:**
- Has primary key (unique identifier)
- Independent existence
- Single rectangle symbol

MOVIE Strong Entity

- Movie_ID: 101 (Primary Key)
- Title: "Inception"
- Release_Year: 2010
- Duration: 148 minutes

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is a Weak Entity? How is it different from Strong Entity?**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is a Weak Entity? How is it different from Strong Entity?**
- **Answer:**
- **Weak Entity** = Cannot exist without another entity (owner entity)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What is a Weak Entity? Why is it called "weak"?
- Why "Weak"?
-

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What is a Weak Entity? Why is it called "weak"? Weak Entity = Entity that cannot be uniquely identified by its own attributes alone
- Why "Weak"?
- 1. No independent key - Doesn't have its own primary key
- 2. Existence dependent - Cannot exist without owner/parent entity
- 3. Needs owner's key - Uses owner's key + partial key for identification

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Aspect	Strong Entity	Weak Entity
Key		
Existence		
Symbol		
Relationship		

SERIES |═══Contains◈═════════

Aspect	Strong Entity	Weak Entity
Key	Own primary key	Partial key (needs owner's key)
Existence	Independent	Dependent on owner
Symbol	Single rectangle □	Double rectangle ▭
Relationship	Regular diamond ◇	Identifying relationship ◈

SERIES ═══Contains◈═══ EPISODE

CUSTOMER ═══Has◈═══TRANSACTION

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- Can a Weak Entity participate in other relationships? Give example.
- Yes! Weak Entity can have multiple relationships
- One Identifying Relationship (with owner)
- Other Regular Relationships – Optional

•WATCHLIST_ITEM ◈ Has ◈ USER (Identifying - required for existence)
•WATCHLIST_ITEM ◇ References MOVIE (Regular - links to which movie)
•WATCHLIST_ITEM ◇ Added_By DEVICE (Regular - tracks which device added it)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- Can a Weak Entity participate in other relationships? Give example.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What is a Partial Key?

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What is a Partial Key?
- Attribute that partially identifies a weak entity (shown with dashed underline)
- Characteristics:
- Not unique by itself
- Becomes unique when combined with owner's key
- Only exists in weak entities
- Series(Strong) has episodes(Weak)
- Partial Key: Episode_Number (values: 1, 2, 3, 4...) Not unique alone!
- Full Key = Series_ID + Season_Number + Episode_Number

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What is an Identifying Relationship? How is it different from regular relationship?
- Comparison:

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What is an Identifying Relationship? How is it different from regular relationship?
- Identifying Relationship = Relationship that helps identify the weak entity
- Symbol: Double Diamond ◈

- Comparison:

Aspect	Regular Relationship ◇	Identifying Relationship ◈
Symbol	Single Diamond ◇	Double Diamond ◈
Purpose	Connects entities	Connects weak entity to owner
Key dependency	No key dependency	Weak entity's key depends on it
Example	USER Watches MOVIE	SERIES Contains EPISODE

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- Why do we need Weak Entities?

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- Why do we need Weak Entities?
- **Weak Entities are better for:**
- **Reason 1: Natural Dependency**
- MOVIE —Has— REVIEW
- If movie is deleted, its reviews should also be deleted (cascading delete)
- Review doesn't make sense without the movie!

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- How do you read participation constraints? Give examples.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- How do you read participation constraints? Give examples.
- CRITICAL RULE: Participation line is NEAR the entity it constrains
- Double line (==) near entity → THAT entity MUST participate
- Single line (—) near entity → THAT entity MAY participate
- USER ——— Writes ═══ REVIEW (single) (double)
- Reading:
- Single line near USER → User MAY write reviews (optional)
- Double line near REVIEW → Every review MUST be written by a user (mandatory)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What is Participation Constraint? Explain Total and Partial.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What is Participation Constraint? Explain Total and Partial.
- Participation Constraint = Does an entity MUST or MAY participate in a relationship?
- Two Types:
- Total Participation (Mandatory)
- Double Line == Every entity MUST participate in the relationship
- Minimum participation = 1
- Cannot exist without the relationship
- EPISODE ═══ Belongs_To ——— SERIES (double line)
- Every episode MUST belong to a series You cannot have an episode without a series!

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- 2. Partial Participation (Optional)
- Single Line —
- Entity MAY or MAY NOT participate
- Minimum participation = 0
- USER ——— Watches ——— MOVIE (single line)
- A user MAY not have watched any movie yet New user just signed up → watched 0 movies (allowed!)
- Movie MAY not have been watched by anyone New movie just added → 0 views yet (allowed!)
- Can exist without the relationship

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are Simple and Composite attributes? Why Composite?**
- Pr

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are Simple and Composite attributes? Answer:**
- **Simple Attribute** = Cannot be broken down further
- **Composite Attribute** = Made up of smaller parts
- **Why Composite?**
- Provides flexibility in queries
- You might want to search by last name only

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is the difference between Single-Valued and Multi-Valued attributes?**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is the difference between Single-Valued and Multi-Valued attributes?**
- **Answer:**
- **Single-Valued** = One value per entity
- MOVIE: - Title: "Inception" (only one title)
- **Multi-Valued** = Multiple values per entity (shown with double oval ◎)
- Genres: ["Action", "Sci-Fi", "Thriller"]

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are Stored and Derived attributes? Give examples.**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are Stored and Derived attributes? Give examples.**
- **Answer:**
- **Stored Attribute** = Value is directly stored in database
- Join_Date: "2023-05-15" (stored)
- Birth_Date: "1995-03-20" (stored)
- **Derived Attribute** = Calculated from other attributes (shown with dashed oval)
- USER: Age = Current_Date - Birth_Date
  Birth_Date: 1995-03-20 → Age: 29 years

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is a Key Attribute? How is it different from other attributes?**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is a Key Attribute? How is it different from other attributes?**
- **Answer:**
- **Key Attribute** = Uniquely identifies each entity instance (shown with underline)
- **Characteristics:**
- **Unique:** No two entities have same key value
- **Not Null:** Must have a value (cannot be empty)
- Why not title of a movie?

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **Can an entity have multiple key attributes? What is a Composite Key?**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **Can an entity have multiple key attributes? What is a Composite Key?**
- **Answer:**
- **Yes! There are different types:**
- **1. Simple Key** = Single attribute
- User_ID ← Simple Key
- **Composite Key** = Multiple attributes together form key
- **Candidate Keys** = Multiple possible keysAll are unique, but we pick User_ID as PRIMARY KEY

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are the degrees of relationships? Explain with examples.**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are the degrees of relationships? Explain with examples.**
- **Answer:**
- **Degree** = Number of entity types participating in a relationship
- **1. Unary (Recursive) - Degree 1** Same entity type relates to itself
- **Binary - Degree 2** (Most Common) Two entity types in relationship
- **Ternary - Degree 3** Three entity types in relationship
- **N-ary - Degree N** More than 3 entity types (rare)

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are relationship attributes? Give examples.**
- **Answer:**
- **Relationship Attributes** = Properties that belong to the relationship itself, not to any single entity
- **Why Needed?**
- Information meaningful only when entities are related
- Can't belong to just one entity

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What is Cardinality Ratio? Explain all types.**
- **Answer:**
- **Cardinality** = Maximum number of times an entity can participate in a relationship
- **All Types:**
- **1. One-to-One (1:1)**

•One entity A relates to ONE entity B
•One entity B relates to ONE entity A

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **2. One-to-Many (1:N)**
- USER ——1:N—— WATCHLIST_ITEM
- One entity A relates to MANY entities B
- Each entity B relates to ONE entity A
- One user has many watchlist items
- Each watchlist item belongs to one use
- **Many-to-One (N:1)**
- Many entities A relate to ONE entity B
- Entity B relates to MANY entities A
- MOVIE ——N:1—— DIRECTOR
- Many movies directed by one director
- One director directs many movies

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **Many-to-Many (M:N)**
- Many entities A relate to MANY entities B
- Many entities B relate to MANY entities A
- USER ——M:N—— MOVIE (Watches)
- Many users watch many movies
- Many movies watched by many users

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are relationship attributes? Give examples.**
- **Answer:**
- **Relationship Attributes** = Properties that belong to the relationship itself, not to any single entity
- **Why Needed?**
- Information meaningful only when entities are related
- Can't belong to just one entity

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- What are relationship attributes? Give examples.
- Why Needed?

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are the degrees of relationships? Explain with examples.**

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **What are the degrees of relationships? Explain with examples.**
- **Answer:**
- **Degree** = Number of entity types participating in a relationship
- **1. Unary (Recursive) - Degree 1** Same entity type relates to itself
- **2. Binary - Degree 2** (Most Common) Two entity types in relationship
- **3.Ternary - Degree 3** Three entity types in relationship

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- Each movie has a unique MovieID and the database stores information including the title, release year, duration (in minutes), language, description, and age rating. A movie can receive multiple ratings from different users. The rating relationship captures important data such as the rating value (on a scale of 1-5 stars and the date when the rating was given. Each rating is associated with exactly one user and one movie.
- Each movie is directed by one director, but a director may direct multiple movies. The database keeps track of director information including DirectorID (which uniquely identifies each director), first name, last name, and nationality. The movie entity includes a DirectorID as a foreign key to establish this relationship.
- Movies are categorized by genres. The database stores each genre with a unique GenreID, genre name (such as Action, Comedy, Drama, Horror, etc.), and a description. A movie can belong to multiple genres and a genre can be associated with multiple movies. For example, a movie might be classified as both "Action" and "Thriller". For tracking purposes, the system maintains detailed watch history through watch items.
- The system also supports a watchlist feature where users can add movies they want to watch later. A user can add multiple movies to their watchlist, and a movie can appear in multiple users' watchlists. The database tracks the date when each movie was added to a user's watchlist to help users remember when they bookmarked the movie.

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **Entities with TOTAL Participation (MUST participate):**
- **MOVIE** in DIRECTS → Every movie MUST have a director
- **MOVIE** in BELONGS_TO → Every movie MUST belong to at least one genre
- **WATCH_ITEM** in HAS_WATCH_HISTORY → Every watch item MUST belong to a user
- **WATCH_ITEM** in RECORDS → Every watch item MUST record a movie

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

- **Entities with PARTIAL Participation (MAY participate):**
- **USER** in RATES → User may not rate movies
- **USER** in HAS_WATCH_HISTORY → User may not have watched anything
- **USER** in ADDS_TO_WATCHLIST → User may not use watchlist
- **MOVIE** in RATES → Movie may not be rated
- **MOVIE** in RECORDS → Movie may not be watched yet
- **MOVIE** in ADDS_TO_WATCHLIST → Movie may not be in any watchlist
- **DIRECTOR** in DIRECTS → Director may not have movies in database
- **GENRE** in BELONGS_TO → Genre may not have movies yet

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei

Min (Entity 1)	Max (Entity 1)	Relationship	Min (Entity 2)	Max (Entity 2)	Relationship Type
0	M	USER RATES MOVIE	0	M	M:N
0	M	USER HAS_WATCH_HISTORY WATCH_ITEM	1	1	1:M (Identifying)
1	1	WATCH_ITEM RECORDS MOVIE	0	M	M:1
0	M	USER ADDS_TO_WATCHLIST MOVIE	0	M	M:N
0	M	DIRECTOR DIRECTS MOVIE	1	1	1:M
1	M	MOVIE BELONGS_TO GENRE	0	M	M:N

Copyright © 2007 Ramez Elmasr and Shamkant B. Navathei